

Project: Nepal Earthquake Mainshock Identification

Objective

To Separate mainshocks from aftershocks in Nepal's earthquake catalog (magnitude ≥ 5) using the Gardner-Knopoff method.

Data Sources

- USGS earthquake catalog (1978–2025, $M \geq 5$, Nepal region)
- Nepal boundary shapefile
- USGS tectonic plates layer (for map background)

Process

1. **Filter** earthquakes to those inside Nepal.
2. **Sort** events by time (oldest first).
3. **Apply Gardner-Knopoff declustering:**
 - First event is a mainshock.
 - For each later event, check if it falls within the **time and distance window** of any previous mainshock.
 - If yes → aftershock (remove).
 - If no → new mainshock (keep).
4. **Save** the final mainshock list as a CSV file.

Gardner-Knopoff Windows (used in the code)

Magnitude	Time (days)	Distance (km)
5.0–5.4	15	40
5.5–6.4	30	50
6.5–7.4	60	75
≥ 7.5	180	200

Workflow

Load CSV → Extract lat/lon → Filter to Nepal → Sort by time → Apply windows → Save mainshocks.

Results

- Original events inside Nepal: **96**
- Mainshocks after declustering: **~58**

Limitations

- Windows were originally made for California; we enlarged the $M \geq 7.5$ window for Nepal.
- The method assumes aftershocks happen equally in all directions (not always true).

Conclusion

The Gardner-Knopoff method successfully cleaned the Nepal earthquake catalog. The final CSV contains only mainshocks and is ready for mapping and hazard analysis.

Code (Python)

```

import pandas as pd, json, shapefile
from shapely.geometry import Polygon, Point
from math import radians, sin, cos, sqrt, atan2

# Load Nepal shape
sf = shapefile.Reader("nepal.shp")
nepal_poly = Polygon(sf.shapes()[0].points)

# Load earthquakes
df = pd.read_csv("earthquakes.csv")
df['lon'], df['lat'] = zip(*df['geo'].apply(lambda x: json.loads(x)['coordinates']))

# Keep points inside Nepal
df = df[df.apply(lambda r: nepal_poly.contains(Point(r['lon'], r['lat'])), axis=1)]

# Convert time and sort
df['time'] = pd.to_datetime(df['time'])
df = df.sort_values('time')

# Windows
def window(mag):
    if mag >= 7.5: return 180, 200
    if mag >= 6.5: return 60, 75
    if mag >= 5.5: return 30, 50
    return 15, 40

# Haversine distance
def dist(lon1,lat1,lon2,lat2):
    R=6371; dlat,dlon = map(radians, [lat2-lat1, lon2-lon1])
    a = sin(dlat/2)**2 + cos(radians(lat1))*cos(radians(lat2))*sin(dlon/2)**2
    return R*2*atan2(sqrt(a), sqrt(1-a))

# Decluster
mains = []
for _, e in df.iterrows():
    is_aft = False
    for m in mains:
        if (e['time']-m['time']).days <= window(m['mag'])[0] and \
            dist(m['lon'],m['lat'],e['lon'],e['lat']) <= window(m['mag'])[1]:
            is_aft = True
            break
    if not is_aft:
        mains.append(e)

# Save
pd.DataFrame(mains).to_csv("mainshocks.csv", index=False)

```